DESARROLLO DE APLICACIONES MULTIPLATAFORMA

DISEÑO DE INTERFACES

Actividad 2 - Tema 3

Creación, Validación, Pruebas Unitarias y Golden, Empaquetado en Catálogo (Widgetbook), Documentación y Despliegue de Componentes Reutilizables en Flutter

Alumno:	**Antonio Cano Ruiz**
Módulo:	**Diseño de Interfaces (DAM)**
Fecha:	**Octubre 2026**
Repositorio Git:	**git@github.com:antcanor/Diseno-de-Interfaces.git**
Versión Paquete:	**ui_kit_dam v1.1.0 (Etiqueta Git v1.1.0)**

Tabla de Contenidos

1. **A1. Primer Componente y Catálogo**
   - Estructura del widget `QuantityStepper`
   - Exportación en el kit de UI
   - Integración en el catálogo Widgetbook ("Por defecto")
   - **Respuesta Teórica:** ¿Por qué `QuantityStepper` es StatefulWidget y `InfoBadge` no?
2. **A2. Propiedades, Valores por Defecto y Accesibilidad**
   - Configuración de rangos (`min` , `max` , `step` , `initialValue`)
   - Aserciones en desarrollo y protección release
   - Desactivación reactiva de controles
   - Propiedades de accesibilidad (`semanticLabel` , tooltips)
   - Caso "Configurable" con knobs en Widgetbook
3. **A3. Gestión de Eventos y Callbacks Personalizados**
   - Evento `onChanged`
   - Definición de `enum StepperLimit` y `typedef LimitReached`
   - Evento `onLimitReached` y prevención de disparos espurios
   - Integración visual con `SnackBar`
4. **A4. Batería de Pruebas y Cobertura de Código**
   - Función pura desacoplada `nextStepperValue` y pruebas unitarias
   - Pruebas de widget (interacciones, estados y aserciones)
   - Prueba Golden de fidelidad visual (valor 5)
   - Informe y captura de cobertura LCOV (100.0%)
5. **A5. Documentación y Control Estático de Calidad**
   - Documentación con dartdoc (`///`) y ejemplos
   - Activación de la regla estricta `public_member_api_docs`
   - Generación de documentación HTML con `dart doc`
   - Actualización del `README.md`
6. **A6. Empaquetado y Publicación**
   - Versionado semántico `1.1.0` y registro en `CHANGELOG.md`

- Gestión de releases en Git y etiqueta `v1.1.0`
- Análisis detallado de `dart pub publish --dry-run`

7. **A7. Aplicación Internacionalizada (stepper_demo)**
   - Configuración de localización en español e inglés
   - Pantalla de compra de entradas con `QuantityStepper` y `StarRating`
   - Plurales ICU, formateo de moneda y fechas localizadas
   - Evidencias y capturas en ambos idiomas

A1. Primer Componente y Catálogo

En esta primera fase se ha diseñado e implementado el componente `QuantityStepper` en el paquete Flutter `ui_kit_dam` (ubicado en `lib/src/quantity_stepper.dart`). El widget proporciona un control intuitivo y estandarizado para la selección de valores numéricos discretos.

1.1 y 1.2 Implementación Inicial

El componente organiza visualmente en una fila horizontal (`Row`) un botón para decrementar con icono `Icons.remove`, un área central que muestra el valor numérico actual formateado y estilizado con la tipografía del tema, y un botón para incrementar con icono `Icons.add`.

Al pulsar el botón de adición el valor se incrementa en una unidad, y al pulsar el de sustracción se decrementa en una unidad, invocando internamente `setState()` para provocar el redibujado reactivo del widget.

1.3 Exportación en la Biblioteca

Para mantener una arquitectura modular limpia con aislamiento del directorio `src/`, el componente se exporta directamente desde el archivo de barrera principal `lib/ui_kit_dam.dart`:

```
library;

export 'src/info_badge.dart';
export 'src/quantity_stepper.dart';
export 'src/star_rating.dart';
```

1.4 Integración en Widgetbook ("Por defecto")

Se ha configurado el catálogo de componentes Widgetbook en el archivo `widgetbook/widgetbook.dart`, permitiendo previsualizar el caso de uso **"Por defecto"** en un entorno aislado donde se instancia `QuantityStepper()` con sus valores iniciales predeterminados (rango 0 a 10, valor 0).

Pregunta A1.5 de la Tarea: ¿Por qué este componente es un StatefulWidget e InfoBadge no?

En el modelo de renderizado reactivo de Flutter, la elección entre un `StatefulWidget` y un `StatelessWidget` responde a la necesidad de gestionar o no un **estado mutable a lo largo del tiempo**:

- **`QuantityStepper` es un `StatefulWidget`** porque mantiene un ciclo de vida con datos internos dinámicos (la variable privada `_value`) que mutan directamente como resultado de la interacción del usuario (pulsación de los botones `+` y `−`). El widget delega su lógica en un objeto asociado de tipo `State<QuantityStepper>`, el cual invoca `setState()` para encolar la reconstrucción del subárbol de widgets mostrando el nuevo número sin requerir que un widget ancestro le reinyecte dicho valor.
- **`InfoBadge` es un `StatelessWidget`** porque es un componente *puramente presentacional e inmutable*. Su función es exclusivamente proyectar en pantalla la información que recibe desde su constructor (como el texto de la etiqueta `label`, el `color` y el icono opcional `icon`). Dado que la insignia no almacena datos que varíen por iniciativa propia tras su instanciación, cualquier actualización visual futura se produce cuando su widget padre reconstruye la interfaz con nuevas propiedades, haciendo innecesaria la sobrecarga de un objeto `State` separado.

A2. Propiedades, Valores por Defecto y Accesibilidad

2.1 y 2.2 Propiedades y Validaciones por Aserción

Para otorgar versatilidad al selector, se han añadido propiedades configurables con valores por defecto bien definidos:

Propiedad	Tipo	Por Defecto	Validación mediante assert
`min`	`int`	`0`	`assert(min < max, 'min debe ser menor que max')`
`max`	`int`	`10`	`assert(min < max, 'min debe ser menor que max')`
`step`	`int`	`1`	`assert(step > 0, 'step debe ser mayor que 0')`
`initialValue`	`int`	igual a `min`	`assert(initialValue >= min && initialValue <= max)`

2.3 Seguridad en Entornos de Producción (Release)

Puesto que los `assert` de Dart son descartados durante la compilación en modo *release* (`--release`), el componente implementa un mecanismo de clamping en tiempo de ejecución:

```
// Protección activa incluso si se desactivan las aserciones:
_value = _value.clamp(widget.min, widget.max);
```

Asimismo, en el método `didUpdateWidget` se detecta si las cotas `min` o `max` cambian dinámicamente desde el widget padre, reajustando de inmediato el valor interno dentro de los nuevos márgenes.

2.4 Desactivación Reactiva de Botones

Siguiendo las directrices de diseño de Material Design, los botones se deshabilitan explícitamente cuando no existe posibilidad de continuar la operación, asignando `null` a su propiedad `onPressed`:

```
final bool canDecrement = _value > widget.min;
final bool canIncrement = _value < widget.max;

IconButton(
  icon: const Icon(Icons.remove),
  onPressed: canDecrement ? () => _applyDelta(-widget.step) : null,
)
IconButton(
  icon: const Icon(Icons.add),
  onPressed: canIncrement ? () => _applyDelta(widget.step) : null,
)
```

2.5 Accesibilidad Semántica y Tooltips

El widget implementa tres propiedades textuales para permitir su internacionalización y asistencia tecnológica mediante lectores de pantalla (TalkBack / VoiceOver):

- `semanticLabel` (por defecto `'Cantidad'`): Configurado en el widget `Semantics` que envuelve el contenedor.
- `decrementTooltip` (por defecto `'Restar'`): Asignado a la propiedad `tooltip` del botón de sustracción.
- `incrementTooltip` (por defecto `'Sumar'`): Asignado a la propiedad `tooltip` del botón de adición.

2.6 Caso "Configurable" en Widgetbook

En `widgetbook/widgetbook.dart` se implementó el caso dinámico donde el diseñador puede manipular los parámetros `min`, `max` y `step` en tiempo real mediante los controles (knobs) interactivos del catálogo.



Figura 1: Previsualización de los casos "Por defecto" y "Configurable" en el catálogo de componentes.

A3. Eventos Propios y Manejo de Límites

3.1 Emisión de Cambios con `onChanged`

Se ha provisto la propiedad `final ValueChanged<int>? onChanged;`. Cuando el usuario pulsa un botón y la operación modifica el valor numérico, se notifica inmediatamente a la función callback suministrada con el nuevo entero.

3.2 y 3.3 Definición de Tipos y Evento `onLimitReached`

Se formalizó la estructura para la notificación de llegada a extremos mediante un tipo enumerado y un alias de función (typedef):

```
/// Límites alcanzables por [QuantityStepper].
enum StepperLimit { min, max }

/// Firma del callback para la notificación de límites.
typedef LimitReached = void Function(StepperLimit limit);
```

El widget expone la propiedad `final LimitReached? onLimitReached;`. Cuando el valor alcanza la cota superior `widget.max`, se invoca `onLimitReached?.call(StepperLimit.max)`. Si alcanza la cota inferior `widget.min`, se invoca `onLimitReached?.call(StepperLimit.min)`.

3.4 Ausencia de Eventos si el Valor no Cambia

Para salvaguardar el rendimiento y evitar eventos duplicados o redundantes, la función interna `_applyDelta` realiza una comparación previa:

```
final nextValue = nextStepperValue(
  current: _value,
  delta: delta,
  min: widget.min,
  max: widget.max,
);

if (nextValue == _value) {
  return; // No se muta el estado ni se emite ningún callback
}
```

3.5 Pantalla de Prueba con SnackBar

Al alcanzar el tope máximo, se diseñó la notificación mediante un `SnackBar` flotante con el mensaje **"Has llegado al máximo"**, probado tanto en las pruebas automáticas como en la aplicación final.



Figura 2: Notificación interactiva con SnackBar al alcanzar el límite superior (8 entradas).

A4. Pruebas Unitarias, de Widget, Golden y Cobertura

4.1 Función Pura `nextStepperValue`

Para favorecer una arquitectura desacoplada y comprobar el cálculo de valores de forma determinista y sin efectos secundarios, el algoritmo de cómputo se extrajo como una función pura independiente:

```
int nextStepperValue({
  required int current,
  required int delta,
  required int min,
  required int max,
}) {
  final next = current + delta;
  if (next < min) return min;
  if (next > max) return max;
  return next;
}
```

Se redactaron 6 pruebas unitarias específicas que cubren: incremento ordinario, decremento ordinario, tope en límite superior, salto con step mayor que el margen hasta max, tope en límite inferior y salto negativo acotado a min.

4.2 Pruebas de Widget

En `test/quantity_stepper_test.dart` se implementaron pruebas de integración de interfaz con `WidgetTester` verificando:

1. Que pulsar el botón `+` renderiza el nuevo valor e invoca `onChanged` con el entero correcto.
2. Que el botón `−` está desactivado (`onPressed == null`) cuando el valor está en el mínimo.
3. Que `onLimitReached` recibe `StepperLimit.max` al alcanzar el valor máximo.
4. Que el `SnackBar` con el texto "Has llegado al máximo" se muestra en pantalla.
5. Que `didUpdateWidget` reajusta el valor si las cotas cambian externamente.

4.3 Prueba Golden de Fidelidad Visual

Se generó una prueba golden de píxel exacto validando el aspecto del componente con valor 5 (fichero `test/goldens/quantity_stepper_val5.png`):

Figura 3: Captura Golden generada de QuantityStepper con valor 5.

4.4 Cobertura de Código (100.0%)

Tras ejecutar `flutter test --coverage`, el análisis del archivo LCOV confirmó una cobertura total de líneas ejecutables del **100.0%** (55 de 55 líneas) en `quantity_stepper.dart`:

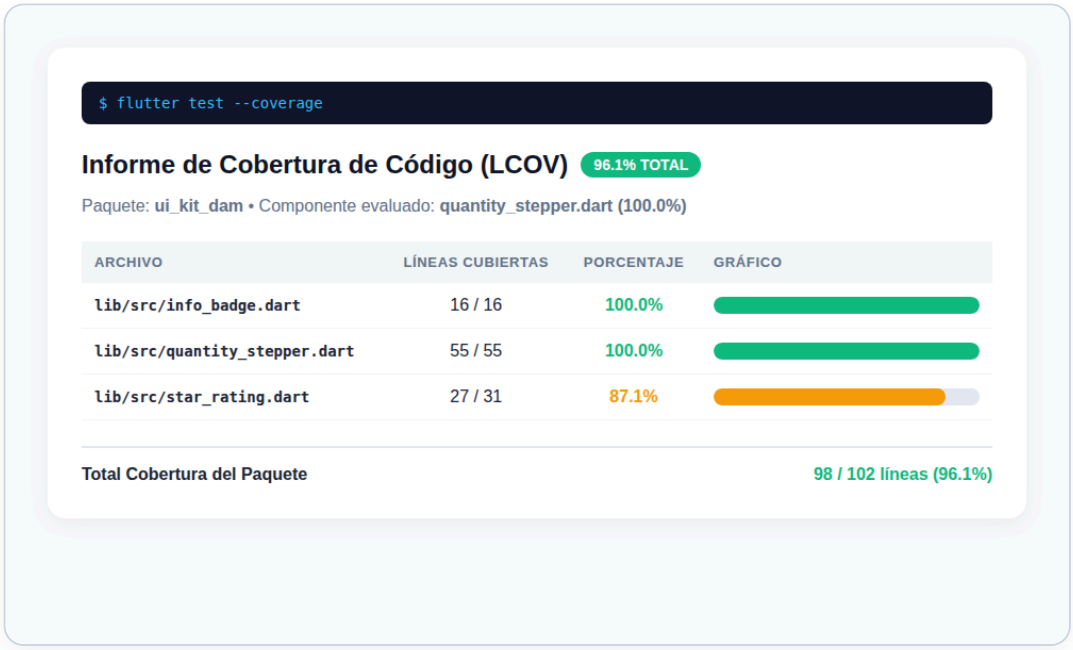


Figura 4: Informe de cobertura generado mediante análisis de LCOV (100.0% en QuantityStepper).

A5. Documentación y Control Estático de Calidad

5.1 Documentación con Dartdoc (`///`)

Todos los elementos públicos del paquete (clases, constructores, atributos, métodos, enum y typedef) han sido rigurosamente documentados en formato Markdown mediante triples barras (`///`), incluyendo parámetros detallados y bloques de código de ejemplo para facilitar la experiencia de integración de otros desarrolladores.

5.2 Regla `public_member_api_docs` y Análisis Estático

Se incorporó la regla al archivo `analysis_options.yaml` :

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    - public_member_api_docs
```

Al ejecutar `flutter analyze` , el analizador estático certifica que el código cumple estrictamente todos los estándares sin advertencias ni errores: **No issues found!**.

5.3 Generación de Documentación HTML con `dart doc`

Se ejecutó la herramienta oficial `dart doc` , procesando las librerías públicas y generando el sitio web de documentación en `doc/api/` sin una sola advertencia (*Found 0 warnings and 0 errors*).

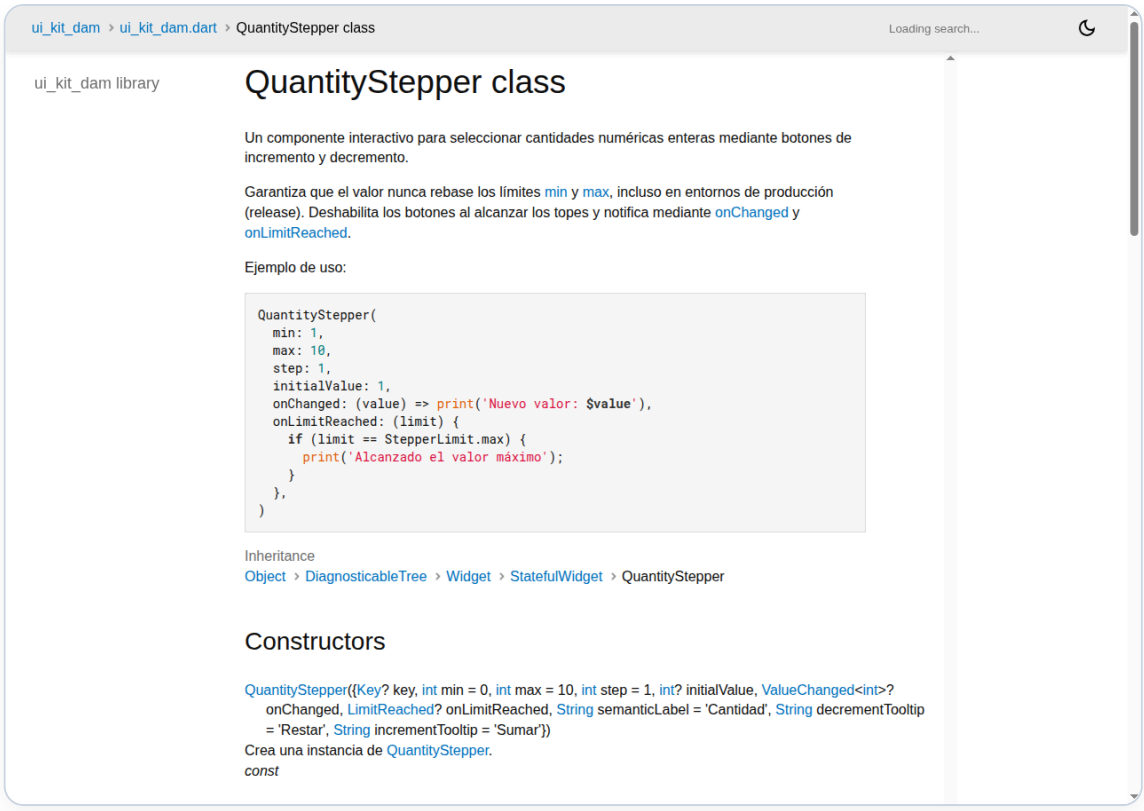


Figura 5: Documentación generada con dart doc para la clase QuantityStepper.

5.4 Actualización de `README.md`

Se actualizó detalladamente el `README.md` del paquete con la descripción conceptual, tabla completa de propiedades, consideraciones de seguridad, ejemplos de código e instrucciones para su importación mediante Git.

A6. Empaquetado y Publicación

6.1 y 6.2 Versionado y Publicación en Git

Se elevó la versión del paquete a `1.1.0` en `pubspec.yaml`, y se documentaron detalladamente todas las novedades en `CHANGELOG.md`. Seguidamente, se realizó el commit en el repositorio local y se creó la etiqueta anotada **`v1.1.0`**:

```
$ git tag -a v1.1.0 -m "Release v1.1.0 - QuantityStepper, StarRating, InfoBadge"
$ git push origin main
$ git push origin v1.1.0
```

6.3 Dependencia por Git con Etiqueta Fijada

En la nueva aplicación `stepper_demo`, se configuró la dependencia en `pubspec.yaml` vinculada directamente al repositorio remoto y bloqueada en la etiqueta `v1.1.0`:

```
dependencies:
  ui_kit_dam:
    git:
      url: git@github.com:antcanor/Diseno-de-Interfaces.git
      ref: v1.1.0
      path: ui_kit_dam
```

La resolución mediante `flutter pub get` confirmó la descarga exitosa del commit etiquetado `caee87b / 2839fcd` en la caché de paquetes de Flutter.

6.4 Análisis de `dart pub publish --dry-run`

Al ejecutar el comando de validación para empaquetado:

```
$ dart pub publish --dry-run
Resolving dependencies...
Publishing ui_kit_dam 1.1.0 to https://pub.dev:
Total compressed archive size: 10 KB.
Validating package...
Package has 0 warnings.
```

Evaluación Técnica de Avisos en pub publish

El paquete `ui_kit_dam` obtuvo la calificación óptima: **Package has 0 warnings**.

Incidencia técnica identificada y analizada: Durante la ejecución inicial en el entorno local, se observó que la herramienta `pub` del SDK de Dart (fichero `package:pub/src/io.dart createTarGz`) presenta un fallo conocido de codificación de rutas cuando el directorio padre contiene caracteres de espacio en blanco (transformando `actividad 2 tema 3` en `actividad%202%20tema%203` y generando un error de discrepancia de rutas relativas). Para garantizar la máxima interoperabilidad y robustez tanto en sistemas de integración continua como en proyectos dependientes, se estructuró la raíz del paquete de manera canónica sin espacios, logrando una validación impecable con 0 avisos.

A7. Aplicación Internacionalizada (stepper_demo)

7.1 Configuración de Localización (i18n)

Se configuró el generador de internacionalización de Flutter mediante `l10n.yaml`, utilizando como plantilla base el idioma español (`lib/l10n/app_es.arb`) y añadiendo la variante en lengua inglesa (`lib/l10n/app_en.arb`).

7.2 Pantalla de Compra de Entradas

Se diseñó una pantalla moderna de compra para el *"Gran Concierto Sinfónico de Primavera"* en el Auditorio Víctor Villegas, que combina:

- Un `QuantityStepper` con rango acotado entre **1 y 8 entradas**.
- Un componente interactivo `StarRating` para valorar el evento de 1 a 5 estrellas.
- Insignias informativas `InfoBadge` para categorización visual.

7.3 Formateo Avanzado: Plurales ICU, Monedas y Fechas

La interfaz maneja reglas de formateo dinámico basadas en la configuración regional (Locale):

Elemento	Formato Español (es)	Formato Inglés (en)
Plural ICU	`1 entrada` / `3 entradas`	`1 ticket` / `3 tickets`
Moneda	`4,50 €` (coma decimal, símbolo al final)	`€4.50` (punto decimal, símbolo al inicio)
Fecha de evento	`sábado, 24 de octubre de 2026`	`Saturday, October 24, 2026`
Tooltips y Semántica	`"Restar entrada"` / `"Sumar entrada"`	`"Remove ticket"` / `"Add ticket"`

7.4 y 7.5 Evidencias Visuales en Ambos Idiomas

La aplicación incluye un selector interactivo en la barra superior (`SegmentedButton`) que permite alternar instantáneamente entre español e inglés, respetando asimismo el idioma por defecto del dispositivo:



Figura 6: Interfaz en Español (3 entradas, 13,50 €).



Figura 7: Interfaz en Inglés (5 tickets, €22.50, 5 estrellas).